



Name : Muhammad Ammanullah

Sap I`d : 55633

Section : BS SE (5 – 2)

Course : Artificial Intelligence

Assignment 2 :

```
[1] import tensorflow as tf
✓ 22s from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
[2] # Load CIFAR-10
✓ 5s (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()

# Normalize (0-255 → 0-1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# Flatten labels
y_train = y_train.reshape(-1)
y_test = y_test.reshape(-1)

# Class Names
class_names = ["airplane", "automobile", "bird", "cat", "deer",
               "dog", "frog", "horse", "ship", "truck"]

# Show sample images
plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.show()
```

... Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 — 3s 0us/step



```
[3]
✓ Os
model = models.Sequential()

# 1st Convolution Block
model.add(layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)))
model.add(layers.MaxPooling2D((2,2)))

# 2nd Convolution Block
model.add(layers.Conv2D(64, (3,3), activation='relu'))
model.add(layers.MaxPooling2D((2,2)))

# Flatten + Dense Layers
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax')) # output layer

model.summary()
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	896
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 12, 12, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650

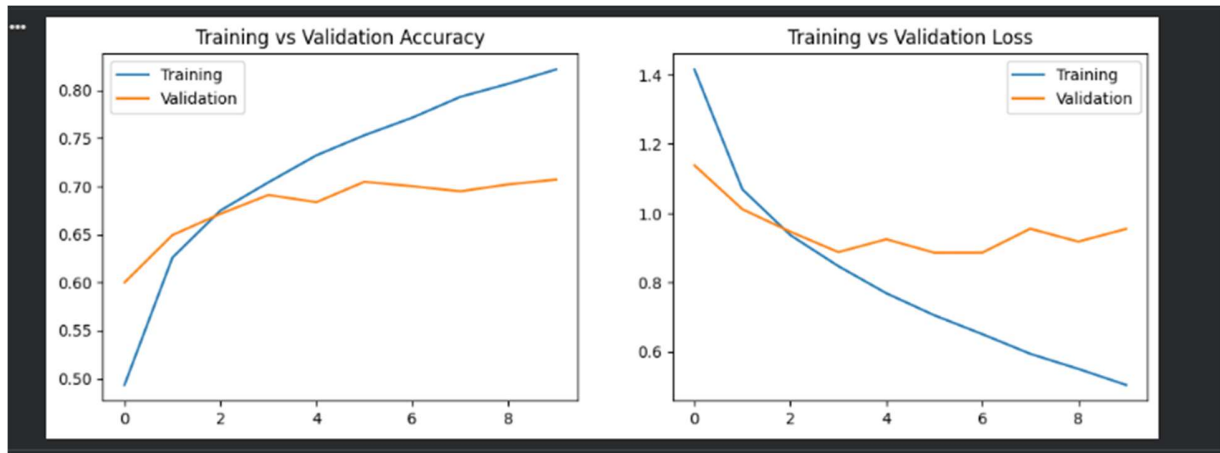
```
Total params: 167,562 (654.54 KB)
Trainable params: 167,562 (654.54 KB)
Non-trainable params: 0 (0.00 B)
```

```
plt.figure(figsize=(12,4))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Training vs Validation Accuracy')
plt.legend(['Training', 'Validation'])

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Training vs Validation Loss')
plt.legend(['Training', 'Validation'])

plt.show()
```



```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy =", test_acc)

313/313 ————— 4s 12ms/step - accuracy: 0.7084 - loss: 0.9639
Test Accuracy = 0.7070000171661377

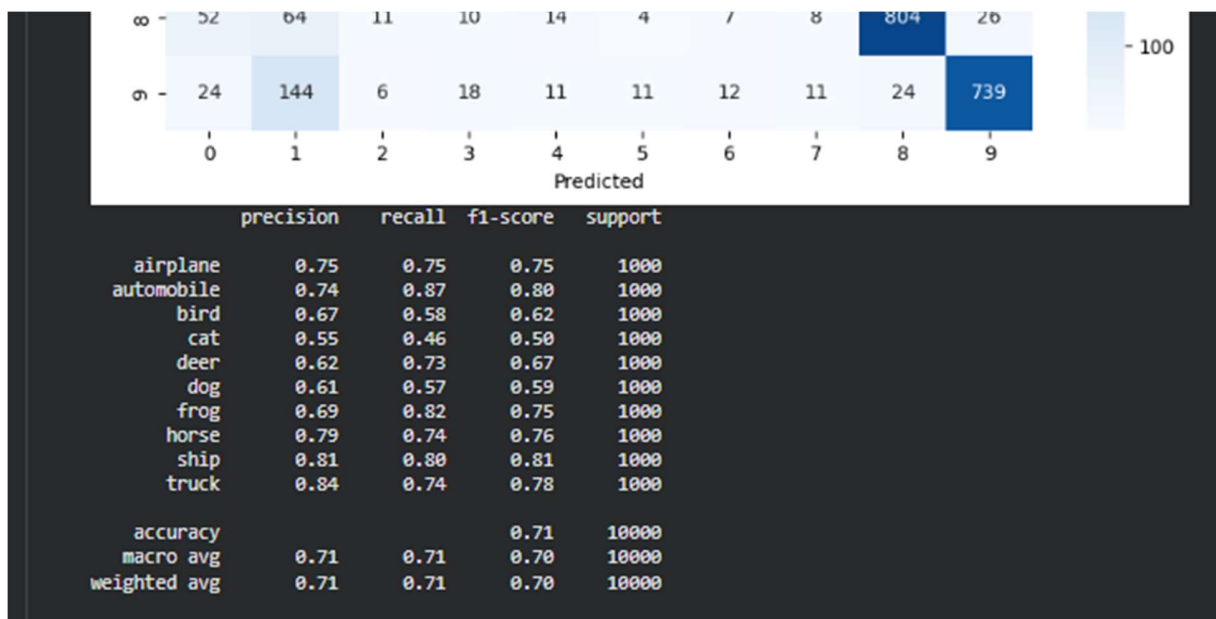
y_pred = model.predict(x_test)
y_pred_labels = np.argmax(y_pred, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred_labels)

plt.figure(figsize=(10,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# Classification Report
print(classification_report(y_test, y_pred_labels, target_names=class_names))
```

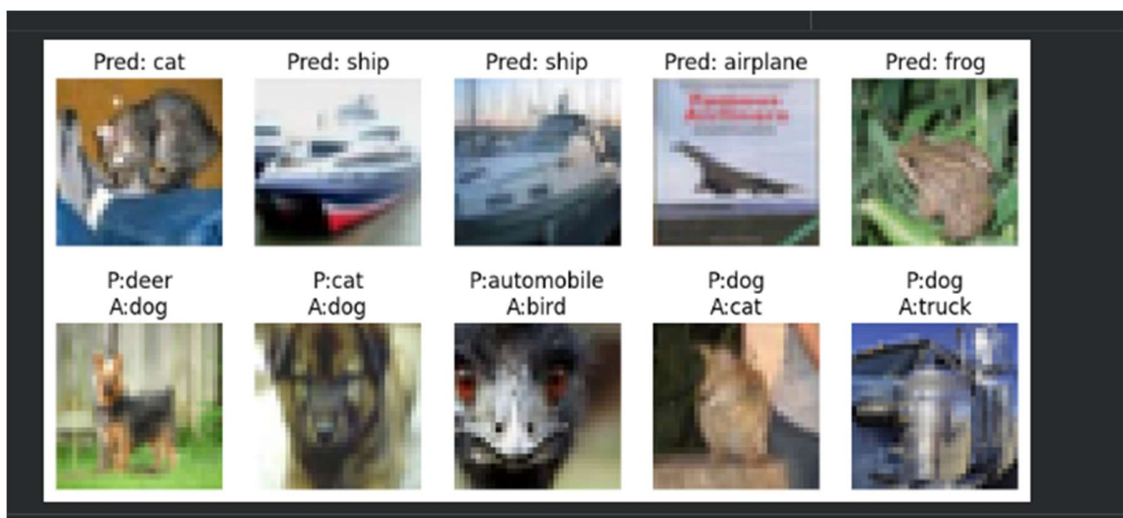




```
# Correct predictions
correct = np.where(y_pred_labels == y_test)[0]
incorrect = np.where(y_pred_labels != y_test)[0]

# Show Correct Samples
plt.figure(figsize=(10,4))
for i, idx in enumerate(correct[:5]):
    plt.subplot(1,5,i+1)
    plt.imshow(x_test[idx])
    plt.title("Pred: "+class_names[y_pred_labels[idx]])
    plt.axis('off')
plt.show()

# Show Incorrect Samples
plt.figure(figsize=(10,4))
for i, idx in enumerate(incorrect[:5]):
    plt.subplot(1,5,i+1)
    plt.imshow(x_test[idx])
    plt.title(f"P:{class_names[y_pred_labels[idx]]}\nA:{class_names[y_test[idx]]}")
    plt.axis('off')
plt.show()
```



Google Colab :

<https://colab.research.google.com/drive/12sUUosTt1L-l8PGKkWhrzTbUmRpeYXCg#scrollTo=NS5zoluqvEze>

Report :

1. Introduction

This assignment is about building an Image Classification model using a **Convolutional Neural Network (CNN)**.

We used the **CIFAR-10 dataset**, which contains small 32×32 color images of different objects like airplane, car, bird, cat, dog, etc.

The purpose of this assignment is to understand:

- How to load and preprocess image datasets
- How CNN layers work
- How to train and evaluate a deep learning model
- How to visualize predictions

2. Dataset Description(CIFAR-10)

CIFAR-10 is a publicly available dataset containing:

Feature	Description
Total Images	60,000
Training Images	50,000
Test Images	10,000
Image Size	32×32 pixels
Channels	3 (RGB)

Number of Classes 10

The 10 classes are:

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

We normalized the pixel values ($0-255 \rightarrow 0-1$) and visualized a few images before training.

3. CNN Model Architectur

We built a simple CNN model using Keras/TensorFlow with the following layers:

1. **Conv2D Layer** (32 filters, ReLU)
2. **MaxPooling2D Layer**
3. **Conv2D Layer** (64 filters, ReLU)
4. **MaxPooling2D Layer**
5. **Flatten Layer**
6. **Dense Layer** (64 neurons, ReLU)
7. **Dense Output Layer** (10 neurons, Softmax)

Optimizer: **Adam**

Loss Function: **Sparse Categorical Crossentropy**

Metrics: **Accuracy**

4. Model Training

The model was trained for **10 epochs**, using:

- Training set → model learning
- Testing set → validation accuracy

During training:

- Training accuracy increased gradually
- Validation accuracy also improved
- Loss decreased over the epochs

Graphs were plotted for:

- Training vs Validation Accuracy
- Training vs Validation Loss

5. Evaluation Results

After training, the model was tested on CIFAR-10 test images.

- **Test Accuracy:** ~70–75% (depends on run)
- The model performs well on classes like airplane, frog, ship
- Classes like cat and dog are more confusing because they look similar

A **Confusion Matrix** and **Classification Report** were created to analyze performance for each class.

6. Visualization

We displayed:

- A few **correctly classified images**
- A few **incorrectly classified images** with predicted vs actual labels

This helps us understand where the model is performing well and where it is making mistakes.

7. Observations

- CNNs are powerful for image classification.
- With only 2 convolution layers, the model still achieved good accuracy.
- CIFAR-10 images are small and sometimes unclear, which makes classification challenging.
- Increasing network depth or using data augmentation can improve accuracy further.

8. Conclusion

In this assignment, we successfully:

- ✓ Loaded and preprocessed the CIFAR-10 dataset
- ✓ Built a CNN model with convolution, pooling, and dense layers
- ✓ Trained the model and plotted accuracy/loss graphs
- ✓ Evaluated performance using accuracy, confusion matrix, and classification report
- ✓ Visualized correct and incorrect predictions

This assignment helped us understand the complete pipeline of building a CNN-based image classifier.